

Load Balancer basato su Consumo Energetico ed Emissioni di CO2

Beatrice Proietti (0381537)
Università Tor Vergata

Abstract—Presentiamo la progettazione e l'implementazione in Go di un load balancer distribuito, il cui obiettivo è quello di instradare il traffico di rete in base all'impatto ambientale dei worker. In particolare, il sistema risulta essere sia carbon-aware sia energy-aware, in quanto sensibile sia al consumo energetico sia all'intensità di carbonio dei worker, i quali sono posti virtualmente in regioni geografiche differenti. Sono state progettate e implementate quattro politiche di instradamento: Round Robin, Least Connections, Green Top-K e Green Least Connections. Poiché la telemetria hardware reale (Scaphandre/RAPL) risulta sistematicamente inaccessibile sotto ambiente virtualizzato, la potenza istantanea è stimata con un modello reattivo al carico. Descriviamo le scelte progettuali per la tolleranza ai guasti (rielezione del leader, eviction dei worker per heartbeat mancati), i trade-off e la metodologia sperimentale adottati per il confronto quantitativo tra politiche.

Keywords—load balancer, leader election, energy, carbon intensity, failure detection, Docker, Go

I. INTRODUZIONE

Il progetto implementa load balancer distribuito di livello applicativo, che instrada richieste HTTP verso un insieme di worker con l'obiettivo di favorire, quando possibile, i worker energeticamente più efficienti e situati in zone a minore intensità di carbonio, senza sacrificare la disponibilità del servizio in caso di guasto di un nodo qualsiasi del cluster. I requisiti che hanno guidato ogni scelta progettuale sono tre: tolleranza ai guasti (il cluster deve restare operativo alla perdita di un nodo LB o di un worker), consapevolezza ambientale (le politiche di routing devono poter pesare l'impatto energetico/carbonico, non solo il carico), e riproducibilità sperimentale (il confronto tra politiche deve avvenire a parità di condizioni offerte). L'assenza di un requisito di consistenza forte sull'ordine delle richieste instradate a differenza, ad esempio, di un registro replicato, orienta la progettazione verso un meccanismo di coordinamento leggero (un solo coordinatore alla volta, riletto alla sua perdita) piuttosto che verso un protocollo di consenso a maggioranza.

II. PROGETTAZIONE DELLA SOLUZIONE

A. Architettura

Il cluster è composto da cinque nodi LB, di cui uno viene eletto leader ed è responsabile sia della raccolta delle metriche riguardanti i worker, sia della propagazione degli aggiornamenti verso i follower. I nodi del load balancer comunicano tramite gRPC tra loro per elezione e aggiornamento dello stato, e con HTTP verso l'esterno, realizzando il paradigma two-ways tipico dei LB di livello applicativo. I nodi di tale cluster instradano il traffico verso sei worker in altrettante zone simulate (SE, FR, GB, IT-NO, DE, PL). Ogni nodo LB conosce staticamente l'indirizzo di tutti gli altri quattro peer e di tutti i sei worker: la piena connessione è garantita dall'avvio, non scoperta gradualmente, coerentemente con una scala di decine di nodi dove il costo di questa scelta resta trascurabile. Questo per garantire massimo rigore scientifico durante la raccolta e l'esaminazione dei dati

riguardanti le prestazioni delle politiche, facendo in modo che ciascun esperimento fosse realizzato a partire dai medesimi presupposti.

Seguendo tali requisiti è stato progettato un load generator i cui parametri, per ogni run e per ogni politica, sono stati impostati sempre allo stesso modo.

B. Elezione Bully del Leader e Responsabilità

L'adozione dell'algoritmo di elezione Bully è stata motivata dalla necessità di designare un singolo nodo coordinatore (Leader) tramite un meccanismo distribuito e dal basso overhead. A differenza di protocolli di consenso più robusti come Raft, il Bully si distingue per la sua spiccata semplicità implementativa: omettendo la complessa gestione della replicazione del log, risulta la scelta ingegneristica ottimale per un sistema che condivide uno stato transitorio e non necessita di strong consistency tra le repliche. Una volta eletto, il Leader assume il controllo esclusivo di operazioni strategiche: è responsabile del fetch periodico delle metriche ambientali dai worker, dell'update dei nodi follower per mantenere allineate le policy di bilanciamento, e della gestione della fault tolerance del cluster tramite l'invio di segnali di heartbeat, la cui interruzione innesci tempestivamente una nuova elezione per ripristinare il sistema in caso di guasto. Il Leader è inoltre responsabile della fault tolerance verso i nodi worker: superata una soglia di heartbeat mancanti, scatta l'eviction e il cluster LB non instraderà più il traffico verso i worker ritenuti offline.

C. Alternative Scartate

- **Raft/Paxos.** Consenso a maggioranza: più complessità implementativa (log replicato, quorum) per un vantaggio — ordine totale delle scritture — che il caso d'uso (routing HTTP stateless) non richiede; inoltre sacrificerebbe disponibilità durante una partizione.
- **Coordinatore centrale statico.** Un singolo nodo designato a priori come LB primario, senza elezione, è un single point of failure esplicito: la sua perdita fermerebbe il cluster senza un meccanismo di recupero automatico.
- **Gossiping.** Meccanismo non adatto alla ristretta quantità di nodi previsti dal sistema.

III. IMPLEMENTAZIONE

A. Telemetria Energetica Sotto Virtualizzazione

Scaphandre stima la potenza istantanea leggendo i registri Intel RAPL esposti dal kernel Linux in `/sys/class/powercap`. Un test diretto (`docker run --privileged --pid host`) mostra che Scaphandre termina in panic sotto virtualizzazione, non trovando i moduli RAPL. Ciò lo rende di fatto non utilizzabile su istanze standard di AWS EC2, dove l'hypervisor non espone quei registri al guest. Il flag `--vm` di Scaphandre non è una soluzione praticabile in questi ambienti,

perché richiede una seconda istanza di Scaphandre in esecuzione sull'hypervisor stesso, non accessibile all'utente su EC2.

B. Modello di Potenza Reattivo

In assenza di Scaphandre, ogni worker stima la propria potenza istantanea come $\text{idleWatts} + \text{dynamicWatts} \times \text{utilizzo}$, dove l'utilizzo è funzione del numero di richieste concorrenti realmente in corso su quel worker (non un valore costante): il modello riproduce la forma tipica della potenza di un package CPU reale (un pavimento a riposo più una quota dinamica che satura), verificato reagire correttamente a un vero test di carico concorrente.

C. Failure Detection

Ogni worker viene interrogato ogni `heartbeat_interval_ms` (2000 ms) con gRPC a timeout stretto (`rpc_timeout_ms`, 800 ms); un worker viene marcato inattivo (eviction) solo dopo `max_missed_heartbeats` (3) mancati consecutivi, non al primo timeout: un singolo timeout isolato è compatibile con una normale fluttuazione di rete o di carico momentaneo, mentre 3 mancati di fila (circa 6 secondi) sono un segnale più robusto di una condizione persistente. Il worker rientra automaticamente al primo heartbeat andato a buon fine.

IV. POLITICHE DI INSTRADAMENTO E VALUTAZIONE

Round Robin e Least Connections instradano ignorando telemetria energetica/carbonica. Green Top-K restringe la scelta al `top_k_percent` (30%) dei worker con minore impatto ambientale stimato, poi ne sceglie uno randomicamente tra questi. Green Least Connections usa invece una funzione di costo continua:

$$\text{costo} = (\text{Watt} \times \text{IntensitàCarbonio}) \times (1 + \alpha)^{(\text{RichiesteAttive})}$$

che sceglie il worker a costo minimo. Con α piccolo il termine esponenziale cresce lentamente e la politica si comporta quasi come "solo verde"; con α grande basta una sola richiesta attiva per far impennare il costo, spingendo verso un comportamento simile a Least Connections puro. α è quindi un compromesso esplicito tra impronta ambientale e concentrazione del carico (e conseguente latenza di coda).

A. Metodologie Sperimentali e Ambiente di Esecuzione

Per confrontare le 4 politiche a parità di condizioni, il generatore di carico usa una sequenza di difficoltà (iterazioni di calcolo per richiesta) generata da un seed fisso: la stessa identica sequenza di richieste, con lo stesso ordine di difficoltà, viene offerta a ciascuna politica, così che le differenze osservate siano attribuibili alla politica e non a una differenza casuale di carico tra un run e l'altro. Ogni combinazione (politica, run) viene eseguita con uno stack completamente riavviato, per eliminare stato residuo tra le misure; sono previste 5 run ripetute per ciascuna delle 4 politiche.

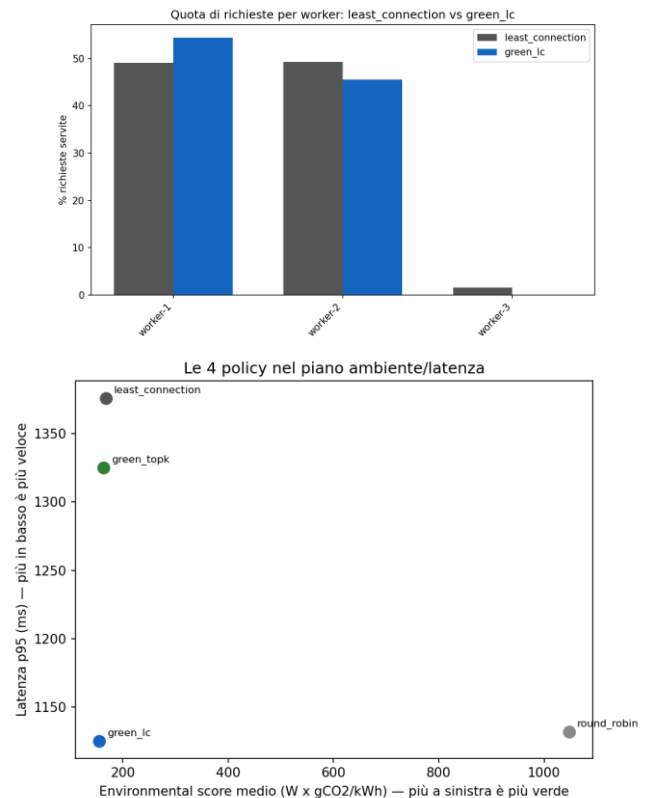
Il deployment su un'istanza Amazon EC2 (AWS Academy Learner Lab) ha richiesto una calibrazione specifica dei parametri di carico: con le impostazioni inizialmente usate in locale (concorrenza 30, difficoltà fino a 4×10^6 iterazioni), la CPU limitata dell'istanza non riusciva a servire in tempo nemmeno i leggeri heartbeat gRPC verso i worker, causando

eviction ripetute e run da 40-50+ minuti con un rate di successo estremamente basso (pochissime delle richieste inviate dal generatore di carico erano effettivamente eseguite); la concorrenza è stata quindi vincolata a circa una richiesta per worker alla volta e la difficoltà massima ridotta di un ordine di grandezza, riportando ogni run in una finestra di circa 3 minuti senza compromettere la validità del confronto tra politiche.

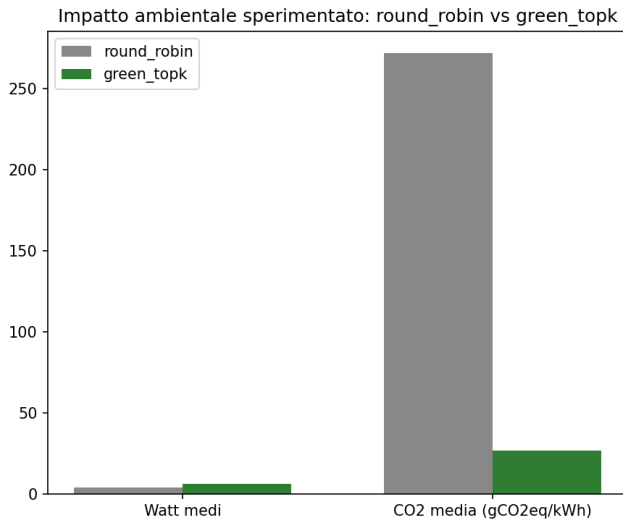
V. RISULTATI

Il sistema soddisfa i requisiti individuati in fase di progettazione: il cluster tollera la perdita di un nodo LB (rielezione Bully verificata, incluso il rientro corretto di un nodo a priorità superiore) e di un worker (eviction e successivo reintegro tramite heartbeat), mentre le quattro politiche di instradamento offrono un ventaglio concreto di compromessi tra prestazioni e impatto ambientale, misurato sperimentalmente e non solo teorico.

I risultati mostrano che l'instradamento carbon-aware è efficace, ma non a costo zero né uniforme tra le due varianti green. Green Top-K, confrontata con Round Robin in condizioni sperimentali pulite, riduce l'impronta di carbonio di quasi un ordine di grandezza, pagando un aumento reale della latenza di coda: un trade-off esplicito, coerente con la scelta progettuale di restringere il routing ai worker più puliti. Green Least Connections, confrontata con Least Connections, mostra nei dati un miglioramento su entrambi i fronti — CO2 e latenza — ma il vantaggio ambientale misurato è probabilmente una sottostima, a causa di un bias di pareggio individuato nell'implementazione di Least Connections (favorisce sistematicamente il primo worker della lista a parità di connessioni attive, indipendentemente dall'ambiente), che ha reso quella particolare baseline artificialmente più verde del previsto.



Il risultato più significativo dal punto di vista concettuale è che il beneficio ambientale osservato è dominato dalla differenza di intensità di carbonio tra zone geografiche, non dalla riduzione del consumo energetico in sé: le politiche green possono addirittura aumentare la potenza media sperimentata (per la maggiore utilizzazione dei worker selezionati) pur riducendo drasticamente la CO2 complessiva. È la conferma sperimentale del fatto che instradare consapevolmente in base alla zona conta più che instradare per minimizzare il solo consumo.



Il limite principale riscontrato — il bias di pareggio di Least Connections — non invalida il sistema, ma invita a ripetere il confronto Least Connections/Green LC con un meccanismo di pareggio randomizzato prima di trarre conclusioni quantitative definitive su quella coppia specifica.

VI. LIMITAZIONI RISCONTRATE

- Il confronto Least Connections/Green LC è parzialmente confuso da un bias di pareggio nell'implementazione di Least Connections.

- La telemetria di potenza è sempre stimata via mock, mai misurata realmente (Scaphandre risultato inutilizzabile sotto ogni virtualizzazione disponibile).
- I vincoli di CPU dell'istanza EC2 hanno imposto un carico di prova ridotto rispetto a quello originariamente previsto.
- La politica Green Least Connections è attualmente basata su un parametro α fissato a priori.

VII. SVILUPPI FUTURI

Si propone di investigare sull'iperparametro α della politica Green Least Connection, esplorandone i valori per adattarlo al sistema, anziché utilizzare un parametro fisso a priori. Inoltre, si propone la ripetizione della valutazione delle politiche Least Connections e Green Least Connections, eliminando il bias da cui era affetta la prima policy col fine ultimo di trarre ulteriori conclusioni sulla bontà della relativa politica green.

VIII. CONCLUSIONI

Il sistema risulta funzionante e coerente con le specifiche: l'elezione Bully è funzionale alla tolleranza ai guasti e alla diffusione delle metriche usate dal sistema. Le quattro policy di routing sono state tutte verificate sperimentalmente. Le politiche green riducono effettivamente la CO2 (di un ordine di grandezza per Green Top-K, meno per Green LC, anche se il risultato è probabilmente sottostimato a causa del bias citato in precedenza). Il costo è un trade-off di latenza per Top-K, non per LC. Il beneficio ambientale risulta provenire dalla scelta della zona, piuttosto che dal risparmio energetico.

REFERENCES

- [1] Hubblo, "Scaphandre," <https://github.com/hubblo-org/scaphandre>
- [2] Electricity Maps, <https://www.electricitymaps.com>
- [3] gRPC / Protocol Buffers, <https://grpc.io>